Advances in Cognitive Systems X (20XX) 1-6 Submitted X/20XX; published X/20XX

TRELLIS: A Cognitive System for Concepts and Chunks

Karthik Singaravadivelan[1,3] KSINGARA3@GATECH.EDU
Pat Langley[1,2] PAT.LANGLEY@GTRI.GATECH.EDU
Christopher J. MacLellan[3] CMACLELL3@GATECH.EDU
[1]Institute for the Study of Learning and Expertise, 2164 Staunton Court, Palo Alto, CA 94306 USA
[2]Information and Communications Laboratory, Georgia Tech Research Institute, Atlanta, GA 30318 USA
[3]School of Interactive Computing, Georgia Institute of Technology, Atlanta, GA 30318 USA

Abstract

Humans routinely combine familiar elements into novel composites whose category and behaviour are inherited from their parts, segmenting a stream of words into phrases, or a sequence of moves into a familiar opening. The resulting structure is often, but not always, hierarchically nested. This ability is better characterized as the joint exercise of two mental abilities, categorical generalization and compositional aggregation, than as either one alone, since it requires both grouping novel elements with classes they have never been observed in and combining them into larger composites that the system has come to recognize. In this paper, we present a theory of chunking that builds on earlier concept-formation work but extends it in new directions. The theory distinguishes between two kinds of taxonomic memory, with one organizing the content descriptions of recurring composites and the other organizing the context descriptions in which they appear, and uses these two organizations together to support parsing, generation, and incremental learning under a single elaboration loop. We introduce TRELLIS, an implemented system that instantiates the theory and demonstrates its behavior on a series of synthetic grammars. The system incrementally parses observations into partonomic structures, generates novel experiences by inverting the same machinery, and absorbs the committed parses back into the two hierarchies one elaboration at a time. We conclude by discussing related approaches to chunking and concept formation, limitations of the implementation, and directions for future research.

1. Introduction

The cognitive systems literature has long sought computational accounts of how humans combine familiar elements into novel composites whose category and behaviour are functions of their parts and the way they are combined. That competence is what philosophy and formal linguistics call *compositionality* (Frege, 1956; Montague, 1973): a strictly representational property, distinct from (though often expressed through) the hierarchical structure that repeated composition produces. A system exhibiting it must be systematic and productive over constituent-preserving representations (Fodor & Pylyshyn, 1988; Chomsky, 1995; Fodor, 1975). Two longstanding cognitive-systems traditions have each picked up part of this competence. *Concept formation* has produced incremental, unsupervised systems that organise instances into taxonomic hierarchies but treats every instance

as a flat attribute–value vector with no internal structure, which makes them compositionally inert. *Chunking* has produced models that aggregate recurring configurations into composites but typically match input exactly and do not generalise across structurally similar instances; they are compositional but not categorical. Neither, on its own, satisfies the systematicity constraint.

We propose a theory that unifies the two by making every element of an experience carry both a *content* description, naming its parts, and a *context* description, naming its surroundings. We build off Cobweb (Fisher, 1987) as our substrate for concept formation, extending it with postulates that introduce composition over its taxonomic hierarchies and support incremental parsing of an experience into chunks and the inverse generation of an experience from memory. The content/context split is our discrete counterpart to the role–filler bindings developed in the connectionist tradition since Smolensky (1990), and the basic-level abstraction at which the system samples and recognises is closer in spirit to the constituent labels Merge composes over than to a flat exemplar code. We instantiate the theory in TRELLIS, which runs parsing, generation, and learning over two Cobweb hierarchies. The paper makes three contributions: (i) a postulate-driven *theory* centred on the content/context split; (ii) a concrete *system* that unifies parsing, generation, and learning under one elaboration loop; and (iii) an *empirical study* on three synthetic context-free grammars and a terminal-vocabulary sweep characterising parse accuracy, generation grammaticality, and incremental absorption.

2. Background and Motivation

The way humans combine familiar parts into novel composites whose category and behaviour reflect those of their parts (Newell & Simon, 1972; Langley et al., 2009) has been studied in two parallel paradigms (Langley, 2025). *Concept formation* targets the categorical generalisation that groups a novel instance with classes whose members it has never seen; *chunking* targets the compositional aggregation that assembles familiar parts into larger recurring composites, a process that often, though not necessarily, produces nested hierarchical structure. We review what each provides and leaves out, then state our theory in Section 3.

2.1 Concepts and the Concept-Formation Tradition

Concepts are categorical, graded mental structures that summarise regularities shared across a set of entities and are organised taxonomically (Rips et al., 2012; Langley, 1996; Rosch et al., 1976; Murphy, 2002). Cobweb (Fisher, 1987; Gennari et al., 1989), the system our theory builds on, sorts attribute–value instances from the root toward a leaf with four restructuring actions (add, create, merge, split) at every node. It has been extended toward information-theoretic objectives (McKusick & Langley, 1991), relational structure (MacLellan et al., 2016), refined basic-level computation (MacLellan et al., 2022), image classification (Lian & MacLellan, 2024), and script-like iterative memory (Matsakis et al., 2023). Every variant inherits a flat attribute–value substrate with no vocabulary for naming relations between instances, so the compositional structure pervading language, music, chess, and scene perception, recurring whether or not the resulting form is strictly hierarchical, sits outside what it can express. The psychological evidence makes the gap sharper: *how* concepts combine is itself a substantive problem with no naive compositional answer. The guppy

effect (Osherson & Smith, 1981), the pervasive over- and under-extension in conjunctions (Hampton, 1987), and the theory-driven character of combination (Murphy & Medin, 1985; Murphy, 2002) all show that combination has to be taken as a position, not assumed away.

2.2 Chunks and the Chunking Tradition

Chunks are structures that denote collections of entities forming a specified configuration, often recursively (Miller, 1956; Chase & Simon, 1973). The compositional competence they target is what philosophy and formal linguistics call *compositionality*: the category of a complex expression is a principled function of its parts and their mode of combination (Frege, 1956; Montague, 1973). Fodor & Pylyshyn (1988) argued that this requires *constituent-preserving* representations, and that the resulting behaviour (*systematic* and *productive* generalisation) is non-negotiable for a cognitive system; Chomsky (1995)'s Merge is the canonical recursive binary combinator that delivers it, and Fodor (1975) is the bridge to cognition more broadly. The chunking idea itself was formalised in CHREST (Gobet et al., 2001) (a descendant of EPAM (Richman, 1991)) and runs through Soar (Laird et al., 1984) and grammar-induction work (Wolff, 1982; Langley & Stromsten, 2000). Adjacent literatures sharpened what compositional structure must support: a role–filler binding primitive (Smolensky, 1990; Plate, 1995), transfer of relational rather than surface structure (Gentner, 1983), and the compositional algebra of Coecke et al. (2010). Lake & Baroni (2023) recently showed the relevant systematicity can be acquired by a standard neural learner under meta-learning pressure rather than built in. Chunking systems in the cognitive-systems lineage nevertheless share a limitation concept-formation systems do not: they compose freely but do not generalise across structurally similar instances. A chunk encoding a particular noun phrase cannot, by its own machinery, recognise a structurally identical one whose component words it has never seen.

2.3 Motivation

The two paradigms have complementary limitations: concepts generalise but do not compose; chunks compose but do not generalise. Langley (2025) argues that they have stayed disjoint for institutional rather than principled reasons, and that a unified theory should treat concept-hood and chunk-hood as two aspects of every representation. We share that diagnosis. Our theory takes Cobweb-style taxonomies as the categorical substrate and adds postulates that build the compositional aspect on top, with the central commitment that the substrate is *two* taxonomies: one indexing elements by what composes them, the other by what surrounds them. The split is our discrete reading of the same structure/content factorisation that runs through role–filler accounts (Smolensky, 1990) and Merge's separation of labels from objects (Chomsky, 1995).

3. A Unified Theory of Chunking

We state our theory as a set of postulates about representation, organization, performance, and learning. Because it builds directly on Cobweb (Fisher, 1987), we first restate Cobweb's commitments in the same format so the chunking postulates of Section 3.2 can continue the same numbering, making the inheritance explicit.

3.1 A Review of Cobweb

3.1.1 Representation Postulates.

Cobweb's representational vocabulary distinguishes the units of experience from the categorical summaries the system builds over them.

R1. Knowledge takes two forms: instances and concepts.

R2. An instance is a set of attribute–value pairs describing a single observation.

R3. A concept is a probabilistic summary of a set of instances, expressed as counts over attribute–value pairs.

Instances arrive from outside the system and are propositional bundles of features, with no internal structure beyond the bag of pairs. Concepts are built up inside the system and are not exemplars or prototypes but counters: each concept holds, for every attribute, a tally of how often each value was observed at instances sorted to it. Predictions over a concept therefore amount to reading off marginal distributions, and concepts themselves can be read as predictive distributions over the attribute vocabulary. These distributions are inherently informative but also inherently *mixed*, since a single value distribution at a node averages over every instance that descended into it. The flatness of the instance representation is what prevents Cobweb from naming relations among instances; we will lift that restriction in Section 3.2.

R4. Cobweb treats the attribute–value pairs within an instance as conditionally independent given a concept, so that each attribute's count distribution at a concept can be considered in isolation.

R5. Each attribute's value distribution at a concept is smoothed by a uniform prior, so that unobserved values still receive a non-zero base-rate probability.

Together, R4 and R5 make per-concept arithmetic tractable and robust. Each attribute is scored separately, an instance's joint log-probability is a sum over attributes, and an unobserved value cannot kill an otherwise-strong match. R4 will also be what lets us route an element's content and context descriptions through two separate hierarchies in Section 3.2 without changing what happens at any individual node.

3.1.2 Organization Postulates.

Cobweb arranges its concepts in a hierarchy that mirrors the categorical structure of its instance set.

O1. Concepts are organised in a taxonomic hierarchy rooted at a single concept that summarises all observed instances.

O2. Each non-terminal node in the taxonomy has two or more children that partition the instances it summarises.

O3. Instances reside as terminal nodes of the taxonomic hierarchy.

The hierarchy is taxonomic: each non-root node refers to a strict subset of its parent's instances, so descent corresponds to specialisation rather than to composition. The partition at each non-terminal node is exclusive, in that an instance is sorted to exactly one child, which makes recognition deterministic given a sorting criterion. Terminal nodes are themselves concepts, each summarising

a singleton set of instances, so the same routing machinery handles intermediate abstractions and individual observations.

3.1.3 Performance Postulates.

Cobweb processes a new instance by descending its hierarchy until it can place the instance among existing concepts.

P1. An instance is processed by sorting it down the hierarchy from the root toward a terminal node.

P2. At each non-terminal node, the instance is routed to the child concept that best fits it under a sorting criterion.

P3. Predictions about unobserved attributes of an instance are read from the concept to which that instance is sorted.

The descent is single-pass: the system commits to one child at each level and never backtracks, which lets Cobweb scale to long input streams in real time but also imparts a residual greedy character that our theory inherits. The sorting criterion at each level is typically category utility, which trades intra-category coherence against inter-category discriminability. After sorting, Cobweb predicts the missing attributes of an instance by reading off the value distributions at the concept where the instance landed, which is what makes Cobweb a *generalising* recogniser rather than a strict matcher.

3.1.4 Learning Postulates.

Cobweb interleaves learning with performance: each instance is sorted and then absorbed into the hierarchy in one pass.

L1. Learning is incremental, piecemeal, and unsupervised, and it is interleaved with performance.

L2. At each non-terminal concept along the sorting path, one of four restructuring actions occurs: add the instance to an existing child, create a new child for the instance, merge two children, or split a child.

L3. The action selected at each step is the one that best improves the evaluation criterion at that node.

L4. Once sorting terminates, the incorporated instance is installed as a new leaf concept along the path it traversed.

Each instance contributes one update during the same descent that sorted it; there is no separate training phase and no class labels are required. The four restructuring actions (add, create, merge, split) cover the space of structural changes a node can make as it absorbs a new instance, and the space is small enough to enumerate exhaustively. The same criterion that drives routing in P2 also drives restructuring here, keeping the system's measure of "good organisation" consistent across recognition and learning. By the time the next instance arrives, the hierarchy already reflects every previous one.

3.2 Postulates for Chunking

The Cobweb commitments above carry directly into our theory; the postulates below extend them, with numbering continuing from Section 3.1 to make the inheritance explicit. The central claim is that a chunking system needs not one but two taxonomies in concert, indexed by complementary kinds of description carried on every element.

3.2.1 Representational Postulates.

Our representational vocabulary starts from the same instance/concept distinction as Cobweb but extends it to distinguish primitive from composite elements and to introduce the content–context structure that every element carries.

R6. Every element has a content description, which names the parts that compose it.

R7. Every element has a context description, which names the discrete relations it bears to surrounding elements.

Carrying two descriptions on every element is what lets the same element be recognised through two independent categorical lenses. The content description is the categorical half that bears on composition. For a composite it names the children it is built from; for a primitive it names nothing at all. The context description is the other half: it names not what the element is made of but where it tends to appear. A primitive's context is its sliding window of neighbours; a composite's context is the same window taken over its boundary positions. R6 in particular is the first move that separates our theory from a single-hierarchy concept-formation system, because every element is now required to carry a part-naming description in addition to its surface features, the very property Fodor & Pylyshyn (1988) argue must hold of a systematic representational substrate. The decision to keep *both* descriptions on every element, rather than collapsing them into a richer attribute set, is also our discrete analogue of the structure/content factorisation that runs through binding-based accounts (Smolensky, 1990): a single bundle of attributes would force a learner to compress role and filler into one categorical signal, whereas two indexed taxonomies prevent the signals from interfering.

R8. Every element is either a primitive or a composite.

R9. A chunk is a composite concept whose instances are themselves built from other primitive or composite elements.

The primitive/composite distinction is exclusive and exhaustive: primitives have no internal structure, and composites are always built from other elements. R9 makes the chunk recursion explicit, in that a chunk's parts are already-classified elements rather than raw features, which keeps the partonomic and taxonomic dimensions of memory cleanly separable. This binary, recursive composition is the simplest member of the family for which Chomsky (1995)'s Merge is the canonical example: a single combinator over two labelled objects, iterated, yields discrete infinity from finite means. The recursion is also what makes the two-hierarchy substrate compatible with the conditional-independence assumption inherited from R4: a content description is itself a bag of attribute–value pairs whose values are concept identifiers, and Cobweb can summarise them without modification.

3.2.2 Organisational Postulates.

These representational claims imply a particular organisation of long-term memory.

O4. Primitives and composites are jointly organised in a partonomic hierarchy.

O5. An experience is a collection of instances and the relationships between them, represented as a set of primitive elements together with a set of discrete relations among those elements.

O6. The system maintains long-term memory in the form of two distinct taxonomic hierarchies: a *content hierarchy* organising the content descriptions of R6, and a *context hierarchy* organising the context descriptions of R7.

The partonomic hierarchy of O4 is what holds a parse together: every committed composite is a node whose children are the elements it was built from. It is distinct from the taxonomic hierarchies of O6, as a partonomy answers "what are the parts?" rather than "what category does this fall under?". Experiences (O5) are the inputs that drive learning, not the long-term memory itself: every experience starts as a sequence of primitives plus the local relations among them, and the partonomy is what the system builds incrementally over those primitives as it parses. O6 is the central organisational commitment of the theory: a chunking system needs not one but two taxonomic hierarchies. Maintaining them separately is our discrete reading of the structure/content factorisation that recurs across compositional accounts (Smolensky, 1990; Coecke et al., 2010). A context-tree path can group noun-like tokens together regardless of whether they appeared as primitives or composites, while a content-tree path can group Det+N-shaped composites together regardless of what surrounded them, and neither signal has to leak into the other.

3.2.3 Performance Postulates.

The system uses its memory in two complementary ways: parsing, which incrementally builds chunks from an experience, and generation, which incrementally unpacks chunks back into the primitives that compose them.

Parsing. **P4.** Parsing is an incremental process of elaborations on an input experience.

P5. A single elaboration generates a set of candidate chunks consistent with the current partial parse.

P6. Each candidate chunk is scored against the content and context hierarchies and gated by a recognition threshold.

P7. The best-scoring candidate that passes the recognition threshold is committed to the partonomic parse tree as a new composite element.

P8. Parsing terminates when no remaining candidate passes the recognition threshold.

Each elaboration extends the current partial parse by exactly one composite. The incremental commitment is psychologically natural, since humans parse sentences word by word rather than top-down from the root, and it fits the Cobweb sorting-with-learning loop, since every elaboration can trigger an update to long-term memory. Candidates are proposals for the next composite the system might commit; the two scoring signals do separate work, with the content score asking whether the candidate's composition pattern looks like one the system has previously consolidated and the context score asking whether its surroundings look like ones that have hosted similar composites

before. The pairing is structurally aligned with role–filler accounts of binding (Smolensky, 1990), with our content slots playing the role of typed positions and the context-tree classifications playing the role of position-conditioned fillers. A candidate must clear the threshold to be admissible, and the best-scoring survivor is committed to the partonomy, which is also the moment at which learning fires (L5–L7 below). P8 lets the system fail gracefully: when nothing on the frontier clears the threshold, the parse ends and whatever remains is returned as a partial parse forest, rather than the system being forced to assert spurious chunks at the boundary of its current competence.

Generation. The generation postulates invert the parsing postulates in a precise sense: where parsing walks bottom-up from primitives, generation walks top-down from a composite seed.

P9. Generation is an incremental process of expansions on a composite seed.

P10. A single expansion produces a candidate leaf consistent with the basic-level concept above the seed's content reference.

P11. The candidate leaf is sampled from the content hierarchy and conditioned on the context hierarchy.

P12. The sampled leaf is committed to the partonomic generation tree as the seed's expansion, and its content children (if any) become new seeds.

P13. Generation terminates when every expanded element is a primitive.

Each expansion produces one new element from the seed, mirroring the incremental commitment of P4. The basic-level constraint of P10 plays the same role for generation that the recognition threshold plays for parsing: it anchors the sampling step at a well-supported level of abstraction rather than at an idiosyncratic deep leaf. The basic-level concept (Rosch et al., 1976) is the level at which the categorical structure is most informative, and sampling there yields neither generic nor idiosyncratic expansions. Conditioning the sampled leaf on the context hierarchy (P11) keeps generation distributionally consistent with the experiences the system has seen. A composite seed whose context-tree classification looks like "noun-like position" gets expanded into a noun-like content leaf, not a verb-phrase one; the two hierarchies thus do the same dual work in generation that they do in parsing. Because each sampled leaf's content children are themselves references into the context hierarchy (R9), every expansion produces valid seeds for the next recursive call, and the procedure terminates when every leaf of the generation tree is a primitive.

3.2.4 Learning Postulates.

The postulates below describe how parse-committed elaborations are absorbed into the two taxonomic hierarchies.

L5. Learning consists of incorporating the elaborations produced by parsing into the two taxonomic hierarchies.

L6. Each composite committed during parsing is added to both hierarchies (its content instance to the content hierarchy and its context instance to the context hierarchy), while primitives, which carry no content instance, are added to the context hierarchy alone.

L7. Each addition follows the standard sorting-with-learning procedure of the taxonomic hierarchy that receives it.

There is no separate training phase: every commit during the parsing loop is immediately followed by an update to memory, so parsing and learning interleave one elaboration at a time. This is the same interleaving commitment Cobweb makes in L1, now applied to the two-hierarchy substrate. L6 routes each commit asymmetrically into the two taxonomies: composites contribute to both, which makes the content hierarchy a record of actually-composed structure rather than of surface vocabulary; primitives contribute to the context hierarchy alone, since their content description (R6) is empty. L7 piggybacks on the Cobweb learning postulates L1–L4, so the system inherits the four-action restructuring procedure (add, create, merge, split) without restating it. Together L5–L7 make recognition and learning two faces of a single elaboration step: every chunk the system has consolidated was previously a candidate it recognised, and every candidate it recognises today was previously a chunk it consolidated.

4. The TRELLIS Chunking System

We have implemented a system, TRELLIS, that instantiates the theory of Section 3.2 atop the Cobweb framework of Section 3.1. Theories are abstract and require additional assumptions to make them operational. In this section we describe TRELLIS's representation, the organization of its long-term memory, its parsing and generation processes, and the way it absorbs the results back into memory, referring to the postulates wherever a choice can be traced to a theoretical commitment and to Appendix B for the exact formulas used by the implementation.

4.1 Representation

The TRELLIS architecture realizes the representational commitments of Section 3.2. To make those postulates concrete, the system adopts a specific encoding for primitives, composites, and the chunks that result from parsing them. The notation resembles attribute–value representations used elsewhere in the concept-formation literature, but elements are constructed and consumed in a distinct way.

Every element in TRELLIS carries two attribute–value descriptions: a *content instance* and a *context instance*. The content instance names the parts that compose the element; the context instance names the neighbours that surround it. The two are kept separate, and each is routed to its own hierarchy in long-term memory. Primitives are atomic and have nothing to enumerate, so they carry only a context instance. Composites carry both, as required by R8 and R9. A composite's content instance contains two attributes naming its left and right children. These attributes store identifiers that point into the context hierarchy, where each child has already been classified. A composite therefore always describes its parts in the same vocabulary as the rest of the system's elements. Figure 1 illustrates the two-instance representation for a simple noun-phrase chunk appearing in a sample sentence.

Bag-of-concepts encoding. Both kinds of instance follow what we call a *bag-of-concepts* convention. The value associated with every attribute is not a single symbol but a small bag (a counter)

over concept identifiers, optionally weighted by position. A primitive's context instance populates its left- and right-window attributes with weighted bags whose entries are the concept-hierarchy identifiers of the immediately neighbouring words, and a composite's content instance populates its left and right slots with bags whose entries are the context-hierarchy identifiers of its left and right children.

The rationale for the bag, rather than a single concept identifier, is that Cobweb concepts are themselves *predictive but mixed*. A concept's count distribution over an attribute summarises every instance that descended into the node, so reading the most-likely value gives a confident prediction but also throws away the alternatives that the node is genuinely uncertain between. By keeping the bag we keep the predictive structure intact: the top-k ancestor choices for a neighbour are precisely the distribution the context hierarchy assigns to it, and treating them as a small mixture lets the downstream hierarchy compose composites whose parts straddle a clean category boundary (a Det that is also halfway to a Pronoun in distribution; an N that is borderline NP-internal vs. NP-external) without first being forced to commit. The bag is what makes the second hierarchy's predictions usable as the first hierarchy's input.

Because every attribute value is a bag over concepts rather than a raw surface symbol, the representation is uniform across primitives and composites and across depths of the partonomy: a word, a noun phrase, and a full clause are all encoded as collections of concept-references, and the Cobweb procedure (R4, R5) sees the same conditional-independence structure in each case. The bag-of-concepts convention is also what licenses the recursive content-reference structure of R9: a composite's parts are always already-classified concepts, so the bag at each content slot is a bag over the same vocabulary the rest of the system uses.

Content-reference remapping. The content-instance bag-of-concepts vocabulary changes over the lifetime of a run, because the context hierarchy itself restructures as new instances are absorbed. A composite committed at training time t_1 may refer to a context-tree leaf that, by training time t_2, has been split, merged, or absorbed into a deeper subtree, leaving its old identifier dangling. The TRELLIS implementation handles this with a top-k ancestor pool: each content-reference is canonicalised to a recent ancestor of its target leaf, and whenever the context tree restructures, the encoder pushes a fresh value-remap binding so that the corresponding bag entries in every stored content instance are rewritten on the fly to the new canonical identifier. The pool depth and the top-k width are implementation parameters (we use depth 4 and $k = 7$ for all paper results); they do not affect what the theory commits to, only how the system maintains coherence between the two hierarchies as they grow. Without the remap, the content hierarchy would gradually accumulate dead references to context-tree concepts that no longer exist, and the recursive content-reference structure R9 commits to would break down.

4.2 Organization

TRELLIS organizes long-term memory into two Cobweb taxonomies, as required by O6. Both inherit the commitments reviewed in Section 3.1, but they play complementary roles.

The content hierarchy classifies the content instances of composites and captures the regularities that govern how chunks are built. Because a composite's content instance refers to its children

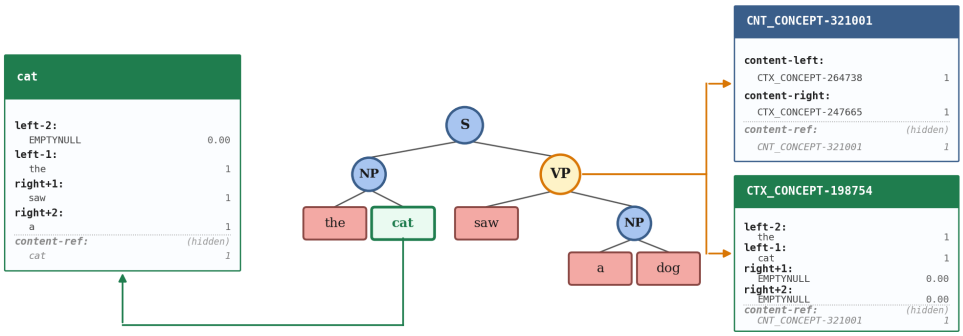

Figure 1. Parse of "the cat saw a dog" showing a primitive ("cat", green) and a composite ("saw a dog", amber) with their instances. Each box header is the long-term-memory identifier under which the instance is classified: a primitive's classifier is the surface word itself (so `cat` appears both as the header and as the `content-ref` of its context instance), while the composite carries a content-tree identifier (`CNT_CONCEPT-*`) for its content instance and a context-tree identifier (`CTX_CONCEPT-*`) for its context instance. Each context instance has four slot attributes (`left-2`, `left-1`, `right+1`, `right+2`) plus a `content-ref` row (dashed italic) carried as metadata but ignored during context-hierarchy routing.

by identifiers drawn from the context hierarchy, similarity in the content hierarchy is grounded in similarity of parts in the context hierarchy. Inspection of a trained content hierarchy typically reveals that composites with structurally similar children, such as an adjective phrase followed by a noun and a determiner followed by a noun, occupy nearby positions in the hierarchy, reflecting their shared role as constituents that can fill an NP slot.

The context hierarchy classifies the context instances of every element the system has encountered, treating each as a singleton defined by its surroundings. Every element has a context instance, so every element appears here; only composites also appear in the content hierarchy, which follows from R8. The context hierarchy is responsible for grouping elements that play interchangeable distributional roles: a noun and an adjective phrase, for instance, will be sorted together in the context hierarchy because they appear before the same kinds of right-neighbours and after the same kinds of left-neighbours. Figure 2 shows a subtree of a trained content hierarchy alongside a subtree of the corresponding context hierarchy, and calls out regions where AdjP and N occupy the same context-hierarchy subtree while the recursive AdjP productions Adj+AdjP and the NP-internal Adj+N occupy the same content-hierarchy subtree.

Alongside these two long-term structures sits a transient working structure: the partonomic parse tree for the current experience. The parse tree is a scratchpad on which TRELLIS accumulates its committed elaborations. Each of its nodes references the content concept and the context concept that classify it, allowing both hierarchies to score the parse as it grows and to absorb its nodes as they are committed.

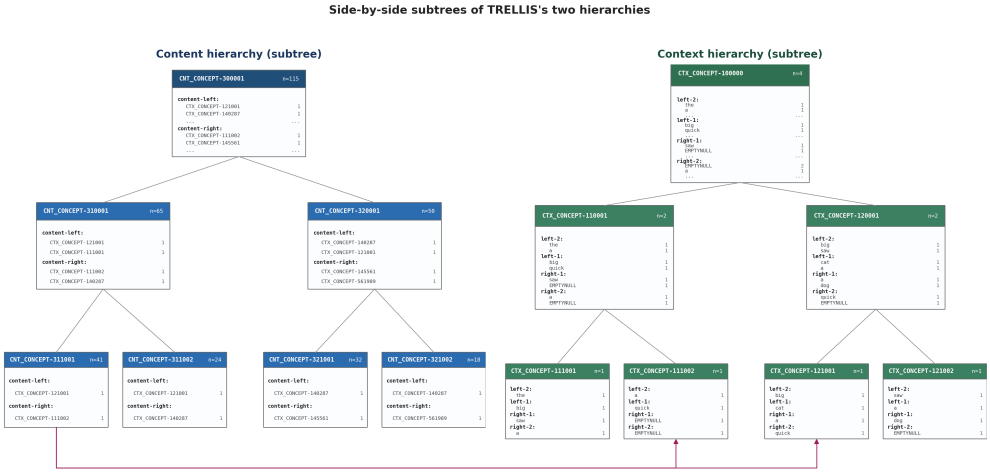

Figure 2. Leaf-level subtrees of the content (left) and context (right) hierarchies, each rooted at a top-level concept and shown down to its leaves. Each node is a Cobweb concept with instance count n. Content-tree attributes `content-left/right` take `CTX_CONCEPT-*` values pointing into context-tree leaves; context-tree attributes `left/right-1,-2` take terminal words, and higher-level concepts sum the distributions of the leaves below them, truncated to the top two values with "…" indicating further entries. Arrows show the cross-hierarchy reference: a content leaf's child slots name the context concepts that classify its children.

4.3 Performance

TRELLIS realizes the performance commitments of postulates P4 through P13 as two complementary processes: parsing, which incrementally builds chunks from a given experience, and generation, which incrementally unpacks chunks back into primitives. The two are described in turn below; the exact formulas for the scoring, threshold, and sampling steps appear in Appendix B.

4.3.1 Parsing.

TRELLIS parses an input experience through a single performance loop, the procedural realization of P4 through P8. The loop consumes primitives and elaborates them into composites. At the start of a parse, the partonomic parse tree is initialized with the primitives of the input; these form the parse frontier from which the first elaboration begins, and the system operates on the frontier from this point on.

Each elaboration forms a candidate chunk for every adjacent pair of elements on the current frontier. A candidate is a hypothesized composite whose left and right children would be the paired elements. TRELLIS evaluates each candidate by routing its content instance through the content hierarchy and its context instance through the context hierarchy, yielding a content-recognition score and a context-recognition score that together express how well the candidate matches the chunks the system has consolidated. In the present implementation the recognition scores are the root log-probabilities of the candidate's content and context instances in their respective hierarchies, which serve as monotonic proxies for the goodness-of-fit of the candidate to each hierarchy.

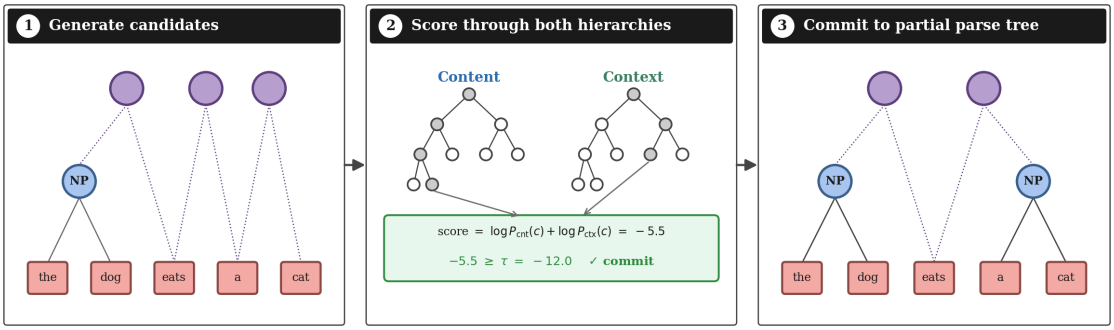

Figure 3. One parsing elaboration: (1) form candidates on the frontier (best in purple); (2) route each candidate through both hierarchies; (3) commit the best one clearing the threshold.

A recognition threshold then separates candidates the system can confidently call familiar from candidates that look like accidents of the current input. Candidates that fail are discarded; the highest-scoring survivor is committed as a new composite. Its two children are removed from the frontier, the composite takes their place, and the next elaboration begins from the updated frontier against an updated long-term memory whose new state is described in Section 4.4. The threshold itself is computed by a climbing-ancestor procedure that walks up the candidate's context-hierarchy path until it finds a level whose support exceeds a tuned minimum; a candidate passes when such a level exists, and fails when the procedure walks all the way to the root without finding one.

The loop repeats until no candidate passes the threshold. Parsing then terminates, and the remaining parse tree is the system's final account of the input: its frontier captures the chunks the system identified, and its internal structure captures the order in which they were assembled. Figure 3 summarizes one round of the loop end-to-end, showing how a candidate is generated, routed through both hierarchies, and committed to the growing parse tree.

4.3.2 Generation.

Generation inverts the parsing loop, as fixed by P9 through P13: where parsing elaborates upward from primitives to composites, generation expands downward from a composite seed back to primitives.

A generation step begins with a seed: a leaf identifier in the content hierarchy carried by some composite still to be expanded. TRELLIS walks the seed upward to its basic-level concept in the content hierarchy and samples a fresh leaf from that basic-level subtree (a representative of the same class as the anchor, but not necessarily the anchor itself), then reads off the sampled leaf's two content references. For each reference the same upward-walk-and-resample procedure is run in the context hierarchy, producing the next pair of seeds. The recursion terminates whenever a sampled leaf is a primitive.

The basic-level concept plays two roles. It fixes the level of abstraction from which the leaf is drawn, and it guarantees that the leaf comes from a class the system has actually consolidated rather than an idiosyncratic exemplar. We approximate it by walking the root-to-leaf path and selecting

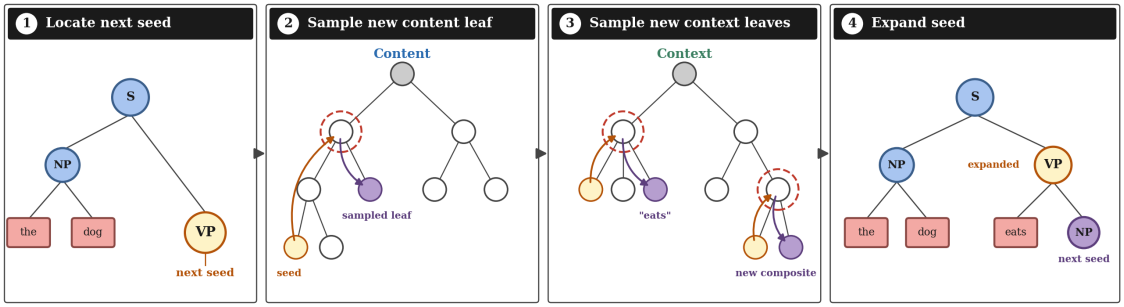

Figure 4. One generation expansion: (1) locate the next seed; (2) walk up to its basic level (dashed red) in the content tree and sample a fresh leaf (purple); (3) repeat in the context tree for each of its two child references; (4) attach the results to the seed. Amber = seed; red dashed = basic level; purple = sampled leaf.

the ancestor whose expected pointwise mutual information (equivalently the KL divergence from the marginal at the root) is largest, as detailed in Appendix B. Because the content children of each sampled leaf are already references into the context hierarchy (R9), every expansion produces valid seeds for the next recursive call, and the procedure proceeds until every branch terminates at a primitive. Figure 4 summarizes one expansion end-to-end.

4.4 Learning

Learning in TRELLIS is the absorption of parse-committed elaborations into the two long-term hierarchies. There is no separate training regime: every commit during the performance loop is followed immediately by an update to memory, so parsing and learning interleave one step at a time. This realizes L5.

When a composite is committed, TRELLIS sorts its content instance into the content hierarchy and its context instance into the context hierarchy, each via the standard Cobweb procedure (L2, L4). The two sorts run independently and in parallel. Primitives encountered during a parse are sorted into the context hierarchy alone, in accordance with L6, since they carry no content instance.

Restricting primitives to the context hierarchy keeps the content hierarchy compositional by construction: every leaf there is a composite whose children are valid references into the context hierarchy. This invariant is what makes the generation procedure of Section 4.3.2 well-defined, since sampling any content leaf yields two valid seeds rather than a malformed reference. Because both hierarchies are updated immediately, candidates produced later in the same parse are scored against a memory that already reflects the chunks committed earlier; over repeated experiences, the two hierarchies converge toward a shared organization of the chunks the system has come to recognize.

4.5 Implementation Details

Two tuned parameters govern the system's behaviour. The first is a uniform Cobweb prior α (R5), which controls base-rate smoothing in every per-attribute value distribution. The second is the

recognition threshold τ, parameterised as a minimum context-hierarchy count, which controls how readily a candidate is judged familiar. Their exact forms, together with the recognition score and basic-level identification, appear in Appendix B; the values used for all experimental results are listed in Section 5.1.

5. Evaluation of TRELLIS

We report a sequence of experiments on three synthetic context-free grammars of increasing complexity. The grammars, their lexicons, and one ground-truth example parse per grammar are collected in Appendix A.

5.1 Setup

Grammars. All three grammars are strictly binary: every nonterminal expansion has exactly two right-hand-side elements. TEST_GRAMMAR_SMALL admits a determiner-noun NP and a transitive VP, with four nouns, four verbs, and two determiners. TEST_GRAMMAR_MED extends it with a right-recursive adjective phrase (AdjP→Adj AdjP, bottoming at Adj+N) and a sentence-level PP-adjunction production S→S+PP. TEST_GRAMMAR_LARGE further adds relative clauses (RelClause) attached recursively at the NP level via NP→NP+RelClause, and an intransitive-V+PP verb-phrase production. Because every production decomposes into exactly two right-hand-side elements, the derivation trees are themselves strictly binary, and gold chunks can be read off them directly without a post-hoc binarisation step.

Training protocol. For each grammar we drew a derivation-annotated corpus and trained TRELLIS incrementally on 200 sentences using the standard parse-and-learn loop, with every composite consolidated through L5–L7 as it is committed. Gold chunks come directly from each derivation tree. Each evaluation point uses a held-out test set of 50 sentences; all metrics are five-seed means with a ± 1 standard-deviation band reported on every curve.

Implementation. The TRELLIS architecture is implemented in Python, with the underlying Cobweb hierarchies provided by a C++ extension. Both hierarchies are instances of the same Cobweb data structure, parametrised only by the attribute vocabulary they classify. The content hierarchy uses attributes for the left- and right-child identifiers along with a small set of complexity annotations that record the structural depth of the composite's children; the context hierarchy uses attributes that name the immediate left- and right-neighbours of the element being classified, plus optional surrounding-window attributes when richer context is available.

System Parameters. A single set of system-parameter values is used for every experiment in this section unless explicitly noted. The context window size is 3 tokens to each side of the element being classified, so each context instance carries six positional slots. The tree-level Cobweb prior is $\alpha_{\text{tree}} = 10^{-4}$, chosen small enough that smoothing does not wash out genuinely sharp value distributions but large enough to keep an unobserved value from killing an otherwise-strong match. The basic-level Cobweb prior is $\alpha_{\text{bl}} = 10$, deliberately large so that the EPMI identification is computed against a smoothed marginal at every ancestor; this keeps the basic-level walk anchored

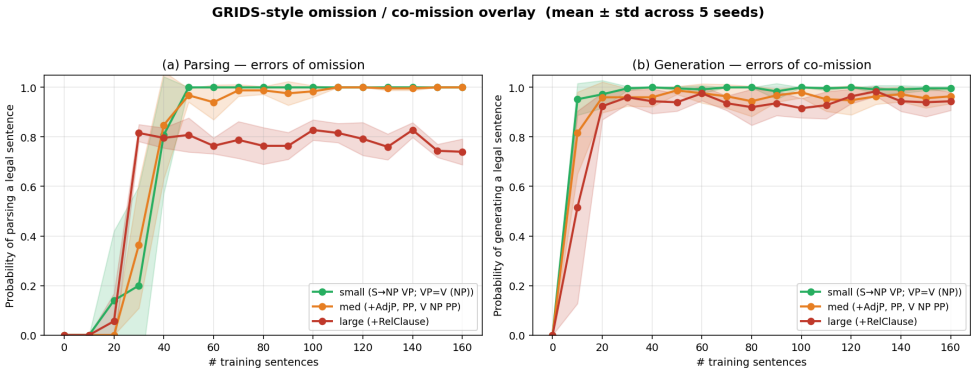

Figure 5. Omission (left) / commission (right) overlay across the three grammars on up to 200 training sentences (5 seeds per grammar). The terminal vocabulary is held constant; only the grammar varies.

at well-supported levels rather than at idiosyncratic deep leaves. The recognition threshold is $\tau = 30$ in absolute mode, so a non-root ancestor on a candidate's context-hierarchy path must have observed at least 30 instances before the candidate can be admitted.

Metrics. We track two complementary error rates over the course of training. *Errors of omission* are the probability that the system fails to recover a legal parse for a held-out sentence, equivalent to a false-negative rate on the parser. *Errors of commission* are the probability that a sampled sentence falls outside the source grammar, equivalent to a false-positive rate on the generator. The omission/commission decomposition was introduced by Langley & Stromsten (2000) in their evaluation of GRIDS, and we adopt it in Sections 5.2 and 5.3 to characterise how parsing and generation behave as the training corpus grows. A separate taxonomy-inspection analysis, showing the empirical POS / chunk-class distribution at the upper levels of both hierarchies on TEST_GRAMMAR_MED, appears in Appendix C.

5.2 Scaling Behaviour Across Grammar Complexity

Figure 5 plots the omission / commission overlay across the three grammars as a function of training-corpus size, with the same 5-seed mean and ± 1 standard-deviation band described above. Parsing omission converges quickly on SMALL and more slowly on MED and LARGE, but generation commission climbs to near 1.0 on every grammar within the first few dozen training sentences and stays there. The asymmetry is consistent with the postulates that drive each process. Parsing depends on per-candidate threshold judgements that drift as the content hierarchy specialises. A candidate that would have passed at training time t_1 may fail at t_2 because the same context-tree ancestor has since been split by an intervening instance, and the climbing-ancestor walk then has to find a deeper still-well-supported level. Generation, by contrast, depends only on structural invariants of the content hierarchy that L6 and L7 keep well-formed by construction: every leaf is a composite whose children are valid context references, so sampling any leaf yields valid seeds.

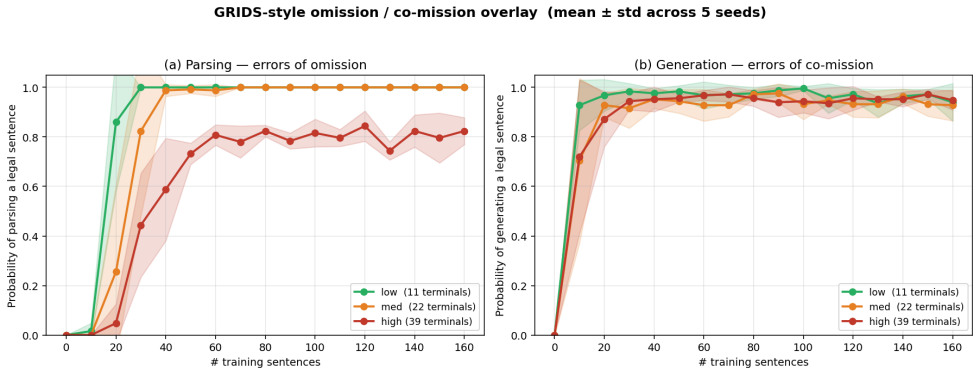

Figure 6. Omission (left) / commission (right) overlay across three terminal-vocabulary sizes on TEST_GRAMMAR_MED (200 training sentences, 5 seeds per size). The grammar is held constant; only the number of terminals per POS varies.

5.3 Scaling Behaviour Across Terminal Vocabulary Size

Figure 6 repeats the overlay across three terminal-vocabulary sizes (low: 11, medium: 22, high: 39 terminals) on TEST_GRAMMAR_MED, holding the productions fixed and varying only the number of terminals per part-of-speech category. Parsing omission is approximately invariant under the vocabulary expansion, with the three curves asymptoting at the same level within the seed-to-seed bands. Generation commission saturates at 1.0 on every vocabulary size. The two-hierarchy organisation absorbs the per-terminal sparsity: the context hierarchy groups distributional equivalents (nouns with nouns, verbs with verbs), so even rare terminals inherit strong context support from their part-of-speech siblings rather than having to be observed many times themselves. The content hierarchy, in turn, sees no growth in its attribute vocabulary at all, since composites continue to refer to children by context-hierarchy identifier rather than by surface word.

6. Related Work

Several bodies of work cover *some* but not all of the functionalities TRELLIS delivers under the postulates of Section 3: two-hierarchy categorical/compositional memory, incremental parsing and inverse generation under a single learning loop, and unsupervised acquisition of constituent structure. We organise this section by what each line of work omits.

6.1 Unified Theories of Concepts and Chunks

The closest precursor to our proposal is Langley (2025)'s recent essay, which argues that a unified theory should treat concept-hood and chunk-hood as two aspects of every representation rather than as two separate kinds. That essay proposes a research agenda of representational, organisational, performance, and learning commitments any unified system would need to satisfy, but does not commit to an implementation. Our theory can be read as one concrete instance of it. Earlier work in the same direction includes the Matsakis system (Matsakis et al., 2023), which extends Cobweb

with iterative script-like additions but lacks the content–context distinction that keeps compositional and distributional structure separable in our framework, and the convolutional Cobweb variant (Lian & MacLellan, 2024), which pairs a chunking-style construction process with a classification-style hierarchy but fixes the construction by architecture and operates on images rather than the sequential partonomic structure that drives our parsing loop.

6.2 Grammar Induction

The problem of inducing a grammar from unannotated sentences asks for the same combination of compositional structure and categorical generalisation our theory addresses. Wolff (1982) introduced an early system that merges frequent adjacent pairs and generalises the resulting categories; its two operations prefigure our two hierarchies, though they are not maintained as separate structures. Langley & Stromsten (2000) introduced GRIDS, which frames grammar induction as Bayesian model selection and evaluates candidates by the omission/commission decomposition we adopt in Sections 5.2 and 5.3. Neural work approaches the problem from a different starting point: supervised parsers such as RNNG (Dyer et al., 2016) and span-based architectures (Stern et al., 2017) learn continuous compositional representations end-to-end but depend on labelled supervision. We view the discrete and neural routes as complementary, since the Cobweb-grounded one supports postulate-level analysis, while the neural one suggests a path to scaling beyond a discrete attribute–value vocabulary.

6.3 Distributional and Neural Language Representations

A third body of work shows that the local context in which a token appears can drive surprisingly rich syntactic categorisation. Distributional embeddings such as Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) and contextualised representations such as BERT (Devlin et al., 2019) cluster words that occur in similar contexts; probes can recover POS tags and constituency structure from BERT's intermediate layers (Tenney et al., 2019). These models extract a context-driven categorical structure of the kind our context hierarchy maintains, but their structure is implicit in trained parameters rather than read off a discrete substrate, which limits the postulate-level analysis a cognitive system can perform on it. Our theory can be seen as a discrete counterpart that keeps the context-driven categorical signal explicit and pairs it with a content-driven one of the same kind.

7. Concluding Remarks

We have presented a postulate-driven theory of chunking that combines the categorical commitments of concept formation with the compositional commitments of the chunking tradition: every element carries both a content and a context description, organised across two Cobweb hierarchies, with one elaboration loop turning them into parsing, generation, and incremental learning. TRELLIS recovers gold chunks at high precision and recall and generates near-perfectly grammatical sentences from memory, without supervision. We note one caveat to the implementation, namely that the basic-level concept is itself an active area of study and our EPMI identification is empirically useful but not formally verified against an independent definition. The two-hierarchy pairing

makes a mutual dependence explicit: categories are defined by the chunks they share, and chunks by the generalisations that appear through categories. The postulate-driven presentation also invites empirical hypotheses about human chunk acquisition testable against psychological data.

The unified theory places chunks and contexts in a mutual dependence neither can do without: chunks are identifiable only because their contexts are similar, and contexts are informative only because they constrain the chunks that fill them. Categories are accordingly defined by the chunks they share, and chunks by the generalisations that appear through categories. Treating this dependence as the foundational commitment offers both explanation and grounds for testing human hypotheses, situating the theory inside the same evidence developmental and psycholinguistic studies have gathered for decades.

Two future endeavors follow. *Unsupervised chunk learning* is the most pressing: TRELLIS still relies on a tuned recognition threshold, and earlier statistical approaches (MDL grammars, mutual-information chunkers, frequency-driven mergers) struggled to say what makes a chunk *good* once superficial recurrence had been exhausted. The theory points toward a chunk-intrinsic answer: a chunk is good to the extent that it is *stable* across the contexts in which it appears, *reusable* as a part of larger chunks, and *predictive* of its own neighbours. An adaptive criterion tracking memory growth should be able to identify such chunks without manual tuning. The second is to push the substrate *beyond sentences* along macro and micro axes: upward into paragraphs and discourse units, downward into letters and phonemes, with the basic-level frontier shifting as more abstract chunks accumulate. The single left/right discrete relation used in TRELLIS should likewise generalise to a broader vocabulary of discrete relations, such as gridlike neighbourhoods, compass directions, or temporal sequences, without changing the rest of the postulates. The unification is one the cognitive systems literature has gestured toward for decades but rarely committed to in implementation; the theory and TRELLIS together establish it as a viable foundation.

Acknowledgements

We would like to thank Zekun Wang, Xin Lian, Kyle Moore, and friends at the Teachable AI Lab for discussions.

References

Chase, W. G., & Simon, H. A. (1973). Perception in chess. *Cognitive Psychology*, *4*, 55–81.

Chomsky, N. (1995). *The minimalist program*. Cambridge, MA: MIT Press.

Coecke, B., Sadrzadeh, M., & Clark, S. (2010). Mathematical foundations for a compositional distributional model of meaning. *Linguistic Analysis*, *36*, 345–384. ArXiv:1003.4394.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (pp. 4171–4186). Minneapolis, MN.

Dyer, C., Kuncoro, A., Ballesteros, M., & Smith, N. A. (2016). Recurrent neural network grammars. *Proceedings of NAACL-HLT 2016* (pp. 199–209). San Diego, California.

Fisher, D. H. (1987). Knowledge acquisition via incremental conceptual clustering. *Machine Learning*, *2*, 139–172.

Fodor, J. A. (1975). *The language of thought*. Cambridge, MA: Harvard University Press.

Fodor, J. A., & Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, *28*, 3–71.

Frege, G. (1956). The thought: A logical inquiry. In *Mind*, volume 65, 289–311. Originally published 1918.

Gennari, J. H., Langley, P., & Fisher, D. H. (1989). Models of incremental concept formation. *Artificial Intelligence*, *40*, 11–61.

Gentner, D. (1983). Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, *7*, 155–170.

Gobet, F., Lane, P. C. R., Croker, S., Cheng, P. C. H., Jones, G., Oliver, I., & Pine, J. M. (2001). Chunking mechanisms in human learning. *Trends in Cognitive Sciences*, *5*, 236–243.

Hampton, J. A. (1987). Inheritance of attributes in natural concept conjunctions. *Memory & Cognition*, *15*, 55–71.

Laird, J. E., Rosenbloom, P. S., & Newell, A. (1984). Towards chunking as a general learning mechanism. *Proceedings of the Fourth National Conference on Artificial Intelligence (AAAI-84)* (pp. 188–192). Austin, TX: AAAI Press.

Lake, B. M., & Baroni, M. (2023). Human-like systematic generalization through a meta-learning neural network. *Nature*, *623*, 115–121.

Langley, P. (1996). *Elements of machine learning*. San Francisco, CA: Morgan Kaufmann.

Langley, P. (2025). Concepts and chunks in cognitive systems. *Proceedings of the Twelfth Annual Conference on Advances in Cognitive Systems*. Palo Alto, CA.

Langley, P., Laird, J. E., & Rogers, S. (2009). Cognitive architectures: Research issues and challenges. *Cognitive Systems Research*, *10*, 141–160.

Langley, P., & Stromsten, S. (2000). Learning context-free grammars with a simplicity bias. *Proceedings of the Eleventh European Conference on Machine Learning* (pp. 220–228). Barcelona, Spain: Springer.

Lian, X., & MacLellan, C. J. (2024). Convolutional Cobweb: A new approach to image classification through concept formation. *Proceedings of the Eleventh Annual Conference on Advances in Cognitive Systems*.

MacLellan, C. J., Harpstead, E., Aleven, V., & Koedinger, K. R. (2016). TRESTLE: A model of concept formation in structured domains. *Proceedings of the Fourth Annual Conference on Advances in Cognitive Systems* (pp. 131–150).

MacLellan, C. J., Matsakis, N., & Langley, P. (2022). Efficient induction of language models via probabilistic concept formation. *Proceedings of the Tenth Annual Conference on Advances in Cognitive Systems*. ArXiv:2212.11937.

Matsakis, N., MacLellan, C. J., & Langley, P. (2023). Incremental induction of phrase-structure grammars through probabilistic concept formation. *Proceedings of the Eleventh Annual Conference on Advances in Cognitive Systems*. Atlanta, GA.

McKusick, K. B., & Langley, P. (1991). Constraints on tree structure in concept formation. *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence* (pp. 810–816). Sydney, Australia: Morgan Kaufmann.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* (pp. 3111–3119).

Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, *63*, 81–97.

Montague, R. (1973). The proper treatment of quantification in ordinary English. In J. Hintikka, J. Moravcsik, & P. Suppes (Eds.), *Approaches to natural language*, 221–242. Dordrecht: Reidel.

Murphy, G. L. (2002). *The big book of concepts*. Cambridge, MA: MIT Press.

Murphy, G. L., & Medin, D. L. (1985). The role of theories in conceptual coherence. *Psychological Review*, *92*, 289–316.

Newell, A., & Simon, H. A. (1972). *Human problem solving*. Englewood Cliffs, NJ: Prentice-Hall.

Osherson, D. N., & Smith, E. E. (1981). On the adequacy of prototype theory as a theory of concepts. *Cognition*, *9*, 35–58.

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing* (pp. 1532–1543). Doha, Qatar.

Plate, T. A. (1995). Holographic reduced representations. *IEEE Transactions on Neural Networks*, *6*, 623–641.

Richman, H. B. (1991). *Discrimination net models of expertise*. Doctoral dissertation, Department of Psychology, Carnegie Mellon University, Pittsburgh, PA.

Rips, L. J., Smith, E. E., & Medin, D. L. (2012). Concepts and categories: Memory, meaning, and metaphysics. In K. J. Holyoak & R. G. Morrison (Eds.), *The oxford handbook of thinking and reasoning*, 177–209. Oxford, UK: Oxford University Press.

Rosch, E., Mervis, C. B., Gray, W. D., Johnson, D. M., & Boyes-Braem, P. (1976). Basic objects in natural categories. *Cognitive Psychology*, *8*, 382–439.

Smolensky, P. (1990). Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, *46*, 159–216.

Stern, M., Andreas, J., & Klein, D. (2017). A minimal span-based neural constituency parser. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics* (pp. 818–827). Vancouver, Canada.

Tenney, I., Das, D., & Pavlick, E. (2019). BERT rediscovers the classical NLP pipeline. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 4593–4601).

Florence, Italy.

Wolff, J. G. (1982). Language acquisition, data compression and generalization. *Language and Communication*, *2*, 57–89.

Appendix A. The Synthetic Grammars

Productions and lexicons of the three CFGs used in Section 5; sentences are derived by uniform selection from each production's RHS alternatives.

TEST_GRAMMAR_SMALL. Det+N NP and V+NP VP only.

S	→	NP VP
NP	→	Det N
VP	→	V NP
Det	→	the | a
N	→	dog | cat | man | woman
V	→	runs | sees | likes | chases

TEST_GRAMMAR_MED. Adds a right-recursive AdjP (bottoming at Adj+N) and S-level PP adjunction via S→S+PP.

S	→	NP VP | S PP
NP	→	Det N | Det AdjP
AdjP	→	Adj AdjP | Adj N
VP	→	V NP
PP	→	P NP
Det	→	the | a
N	→	cat | dog | man | woman | park | telescope
Adj	→	big | small | red | quick | lazy
V	→	saw | liked | chased | found | admired
P	→	with | in | on | under

TEST_GRAMMAR_LARGE. Adds relative clauses (NP→NP RelClause) and a V+PP VP. All productions binary.

S	→	NP VP | S PP
NP	→	Det N | Det AdjP | NP RelClause
AdjP	→	Adj AdjP | Adj N
VP	→	V NP | V PP
PP	→	P NP
RelClause	→	RelPro VP
Det	→	the | a | this | that
N	→	book | boy | girl | teacher | robot | apple
Adj	→	tall | curious | blue | ancient | friendly
RelPro	→	who | that | which
V	→	saw | liked | chased | carried | read | admired
P	→	with | without | near

Sample ground-truth parses. Figure 7 shows one feature-rich sentence per grammar with its gold parse.

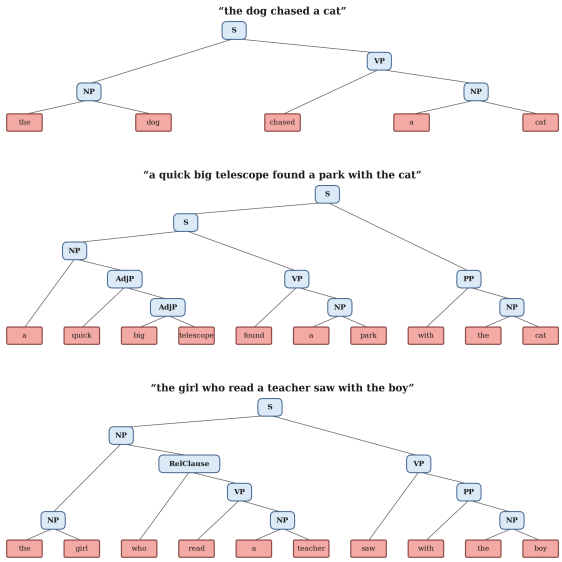

Figure 7. Gold parses for one sentence per grammar. **Top (SMALL):** Det+N NP, V+NP VP. **Middle (MED):** Det+AdjP NP, right-recursive AdjP, V+NP VP, S+PP top split. **Bottom (LARGE):** NP+RelClause subject (inner Det+N), RelPro+VP relative, V+PP matrix VP, P+NP PP.

Appendix B. Implementation Formulas

Recognition score. The Cobweb log-probability of an instance $\mathbf{i}$ under a concept C is, by R4, $\log P(\mathbf{i} \mid C) = \sum_{(a,v)\in\mathbf{i}} \log P(v \mid C, a)$, with the alpha-smoothed distribution $P(v \mid C, a) = (n_{C,a,v} + \alpha_a)/(n_{C,a} + V_a\alpha_a)$ (R5; we use $\alpha = 1/V$ at the tree and $\alpha = 1$ at the basic level).

Recognition scores are root log-probabilities $s_c = \log P(\mathbf{i}_c \mid r_c)$, $s_x = \log P(\mathbf{i}_x \mid r_x)$; the ranking key is $s_x + s_c$, with s_x breaking ties.

Recognition threshold (climbing ancestor). A candidate sorts to a path $\pi = (n_0, \ldots, n_L)$ in the context hierarchy. The climbing-ancestor procedure picks the most-specific ancestor whose support exceeds τ: $n^* = \arg\max_d \{\, n_d : \text{support}(n_d) > \tau \,\}$, where $\text{support}(n) = n^{\text{count}}$ if $\tau \geq 1$ (absolute mode) and $n^{\text{count}}/n_0^{\text{count}}$ if $0 < \tau < 1$ (relative mode, keeping selectivity constant as the tree grows). The candidate passes iff $n^* \neq n_0$.

Basic-level identification (EPMI). The basic-level for a leaf ℓ is the ancestor whose attribute–value distribution is most informative relative to the root marginal, measured by EPMI:

$$\text{bl}(\ell) \;=\; \arg\max_{n \in \text{path}(\ell)} \text{KL}\big(p_{x|n} \,\|\, p_x\big), \quad \text{since } \mathbb{E}_{x|c}[\text{pmi}(x;c)] \;=\; \text{KL}\big(p_{x|c} \,\|\, p_x\big).$$

The implementation walks the root-to-leaf path once.

Appendix C. Taxonomy Inspection

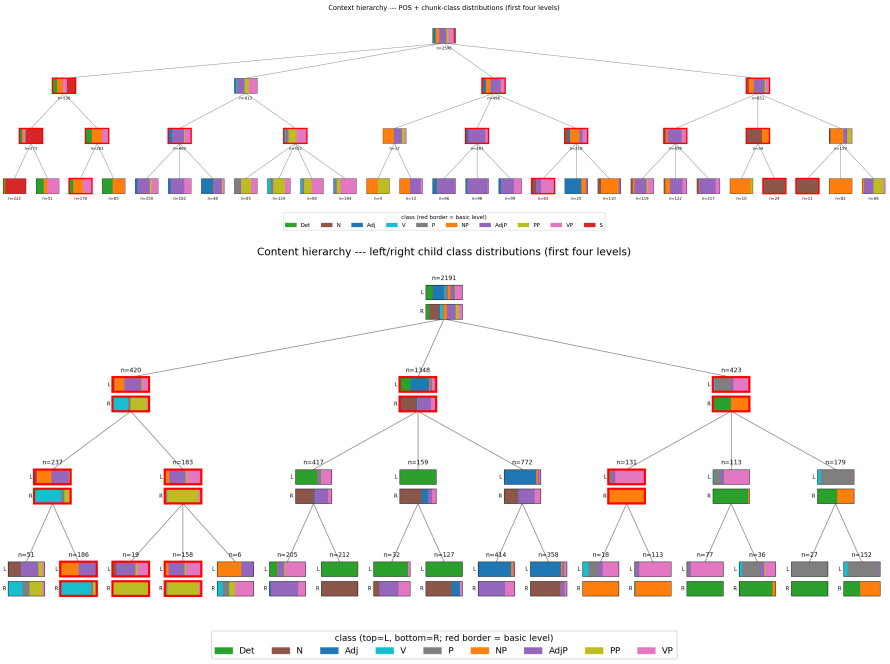

Figure 8. Top four levels (root + three descent rows) of TRELLIS's hierarchies on MED. **(top, context)** Each bar reports the POS/chunk-class distribution of every element (primitive or composite) descending through the node. **(bottom, content)** Each node has two bars: an upper one for the LEFT-child class and a lower one for the RIGHT-child class of every composite descending through it. Red outlines mark basic-level concepts under the EPMI identification ($\alpha = 10$).